

SE/COM S 3190 - Construction Of User Interfaces

Final Project

Published On: October 31, 2025, 5:30 PM CST

Due Date: Before or at the start of your demo (Dec 15–18, 2025)

Contents

1	Overview	2
1.1	Purpose of the Final Project	2
1.2	Learning Objectives	3
1.3	Team Collaboration Guidelines	3
2	Agile & Scrum Framework	3
2.1	Scrum in Context	3
2.2	Issues	4
2.3	Issue Boards	4
2.4	Labels	4
2.5	Milestones	5
2.6	Additional Note	5
2.7	Final Timeline Overview	5
2.8	What you already know!	6
3	Software Requirements	6
3.1	Final Application Expectations	7
3.2	Functionality and Quality Benchmarks	7
3.3	Optional: AI Features – Extra Credit	7
3.4	Technical Requirements	8
3.5	Views and Pages	9
3.6	Folder and File Structure	9
3.7	Use of JSON and External Files	9
4	Documentation Requirements	10
4.1	Purpose and Overview	10
4.2	Format and Submission	10
4.3	Suggested Structure	10
4.3.1	Cover Page	10
4.3.2	Table of Contents	11
4.3.3	Introduction	11
4.3.4	Project Description	11
4.3.5	File and Folder Architecture	11
4.3.6	Code Explanation and Logic Flow	11
4.3.7	Web View Screenshots and Annotations	11
4.3.8	Installation and Setup Instructions	12
4.3.9	Contribution Overview	12
4.3.10	Challenges Faced	12
4.3.11	Final Reflections	12
4.4	Evaluation Criteria	12
5	Video Requirements	13
5.1	Video Duration and Format	13
5.2	Recording Guidelines	13
5.3	Content Checklist	13
5.4	Submission Requirements	14
5.5	Evaluation Criteria	14
6	Live Demo Presentation Requirements	15
7	Final Project Evaluation Rubrics	17
7.1	Software Component Rubric	17
7.2	Technical Report Rubric	17
7.3	Video Rubric	18
7.4	Demo Rubric	18
8	Final Project Points	19
9	Submission Details & Deadline	19
10	Grade Release and Review Window	19

This is a team project. Please review the rubric carefully, as points will be awarded for effective communication and collaboration. Ensure tasks are evenly distributed between both members. Plan accordingly and complete the project the right way.

If you need help, try office hours—the schedule is on *Canvas*. Lots of help is also available through the Piazza discussions. Please start the assignment as soon as possible to get your questions answered right away.

1 Overview

This document outlines the detailed requirements, expectations, and grading rubrics for the Final Project in SE/COM S 3190 – Construction of User Interfaces. The Final Project serves as a comprehensive demonstration of each student team’s learning, growth, and practical application of course content throughout the semester. It reflects not only your technical competence in full-stack development and version control but also your ability to collaborate effectively, communicate ideas, and deliver a polished and meaningful solution to a real-world or fictional problem.

The Final Project is structured to replicate a professional development cycle—starting with project proposals, followed by planning and modular implementation, and ending with final documentation, and presentation. The project is carried out in teams of two and incorporates both individual accountability and collaborative execution.

Teams are expected to demonstrate strong fundamentals in front-end and back-end development, employ thoughtful design strategies for UI/UX, build a scalable system architecture, and document their technical process through detailed planning artifacts and reports.

1.1 Purpose of the Final Project

- To demonstrate mastery of web development skills including React, Node.js, Express, and database integration (MongoDB or MySQL)
- To design and implement a production-ready Single Page Application (SPA).
- To simulate the lifecycle of a real software engineering project: ideation, development, debugging, deployment, and delivery
- To promote collaboration, peer learning, and real-time problem solving in a team setting
- To develop written and verbal communication skills through structured documentation, live demonstrations and retrospectives.
- To apply Agile methodologies and the Scrum framework for iterative development using GitLab’s project management tools such as issues, issue boards, milestones, and labels.

1.2 Learning Objectives

By completing this project, you will:

- Apply theoretical concepts into a hands-on, tangible full-stack software solution
- Gain experience designing system architectures with clear separation of concerns
- Improve front-end design with responsive layouts and user interaction flows
- Implement backend systems that include route handling, middleware, and database queries
- Integrate APIs for CRUD functionality and asynchronous data operations
- Understand security, validation, and error-handling in production-ready applications
- Communicate technical processes through written documentation and oral presentation

1.3 Team Collaboration Guidelines

- Teams must have exactly two members (assigned on *Canvas Groups*)
- Responsibilities must be divided equally — both frontend and backend development tasks should be shared to ensure balanced learning
- Each member must take ownership of specific features, implementing both the frontend and backend components for those assigned features
- GitLab must be used for all project management activities, including maintaining issues, issue boards, milestones, and labels (excluding CI/CD)
- Commits in GitLab must demonstrate consistent, ongoing, and equal contributions from both members throughout the project timeline
- Any collaboration or contribution imbalance must be reported to the TA or instructor before the submission deadline

2 Agile & Scrum Framework

Scrum is a broad development framework, but for this course we'll adopt only selected practices that help teams stay organized and collaborative. The goal is to plan, track, and execute work efficiently using GitLab's built-in Agile tools. **CI/CD pipelines are not required.**

2.1 Scrum in Context

Scrum emphasizes iterative progress through short cycles. While we won't follow full sprint ceremonies, your timeline will simulate sprints through weekly milestones and Mini-Assignments. Each week's progress should show measurable development toward the final deliverable.

2.2 Issues

All features, fixes, or tasks must be tracked as GitLab **Issues**.

- Use clear titles, concise descriptions, and assign ownership.
- Break large tasks into subtasks using checklists.
- Close issues only when completed and committed.

2.3 Issue Boards

Your **Issue Board** acts as a simple Scrum board for tracking progress.

- Each repository will be initialized with a board that includes the columns: **Backlog**, **In Progress**, **Review**, and **Done**.
- Move issues across these columns as work progresses to maintain an accurate, real-time project view.
- The TAs will periodically review each board to evaluate team coordination and individual contributions.

2.4 Labels

Use **Labels** to categorize and filter issues.

- When you receive your project repository, it will already include all standard labels to help organize tasks and track progress efficiently.
- The initialized labels are:
 - **Technical:** frontend, backend, database, API, bug
 - **Progress:** backlog, in-progress, review, completed, blocked
 - **Mini-Assignments:** mini-assignment 1, mini-assignment 2, mini-assignment 3
 - **Documentation:** report, video
 - **Features:** feature 1, feature 2, feature 3, feature 4, feature 5, feature 6, advanced feature
 - **Members:** member 1, member 2, shared task
- Use these labels consistently across all issues to maintain clarity and transparency.
- You may add additional custom labels if needed, but ensure consistent naming and application within your team.

2.5 Milestones

Create Milestones for each Mini-Assignment and the Final Submission.

- Link issues to milestones to track progress.
- Close milestones only when all associated issues are resolved.

These lightweight Scrum tools—Issues, Boards, Labels, and Milestones—keep your project structured and transparent without adding complexity. Use them actively to show teamwork and steady progress throughout development.

2.6 Additional Note

1. The purpose of this project is not just to test your coding skills — it is designed to measure your ability to produce clean, structured, and scalable code that solves a meaningful problem, backed with quality documentation, clear architecture, and logical reasoning.
2. The project requires thoughtful planning, creative thinking, and technical precision. Teams that plan early, divide responsibilities clearly, and iterate frequently will find success. Start early, ask questions often, and aim to build something you're proud of.
3. Incorporating advanced technologies used to solve technical problems won't be penalized. However, if the inclusion of technologies differ from the required interferes with the review of the project at the time it is reviewed, by the TA, the TA can decide whether to penalize the project or not.

2.7 Final Timeline Overview

Proposal — 2%
(Already Submitted)

Final Components — 8%
Due: During Demo (Dec 15–18, 2025)

Software Implementation — 5%

Documentation + Video — 3%

Weekly Mini-Assignments (3 total) begin Nov 10, each due in 1 week. Completion contributes to your final project grade.

Demo — 10%
Dates: Dec 15–18, 2025

Demo scheduling details will be announced by the TAs during the first week of December (after Thanksgiving Break).

2.8 What you already know!

Now that you have almost submitted or completing your proposals, you should have decided between the two project options:

- **Option 1:**

- Scaling your Midterm Project into the Final Project by adding new features and restructuring your existing HTML, CSS, and JavaScript code into a full-fledged React and Node.js application.
- If you chose to build on your Midterm Project, your Final Project will be evaluated in comparison to it. We will specifically look for meaningful improvements and the addition of new functionalities.

- **Option 2:**

- Starting a completely new idea from scratch, which involves designing new wireframes, developing the UI/UX, and implementing the planned features for the application.
- If you started with a new idea, your Final Project should be comprehensive and clearly convey the project's vision, scope, and execution strategy as outlined in your proposal.

3 Software Requirements

The Software component of the Final Project accounts for **5% of your overall grade** and represents the core deliverable of your team's work. It is the final implementation of your planned project and should demonstrate completeness, technical depth, and professional quality.

The development process is supported by a structured sequence of **Mini-Assignments** released starting on **November 10, 2025**, and continuing through **December 14, 2025**. Each Mini-Assignment is designed to help teams incrementally build the application's architecture, UI/UX, backend functionality, and integration logic. If your team consistently completes all Mini-Assignments on time, you will have developed a strong and near-complete foundation for the final submission.

However, your final submission must go beyond baseline functionality. It should demonstrate final refinements, a polished user experience, responsive design, and full integration across all views and features outlined in your project proposal. The project should reflect production-level quality in both design and functionality. A complete **CODE FREEZE** must be in effect by the start of your demo during **December 15-18, 2025**. Any commits or pushes made after this deadline will incur significant penalties, with deductions of up to **50%** of your Final Project grade.

3.1 Final Application Expectations

Your application must:

- Be a Single Page Application (SPA) built in React
- Include user-friendly navigation, responsive layouts, and complete CRUD functionality
- Reflect a clean, professional, and accessible design
- Connect to a database (MongoDB or MySQL) and store persistent data
- Implement full user interaction flows with routing and error handling

3.2 Functionality and Quality Benchmarks

Complete Implementation: The final version must implement 100% of the features defined in the proposal and any required views.

Smooth Navigation: Use React Router (or equivalent) for clean transitions between pages. Avoid reloads or broken navigation flows.

Visual Design: Your project's UI must demonstrate a reasonable level of effort and intentionality. While you are free to determine the aesthetic style, applications with minimal styling or placeholder content will be penalized. Tools like Tailwind CSS, Bootstrap, and custom CSS modules are welcome.

Responsiveness: Ensure that your interface adapts appropriately to different screen sizes, including desktop, tablet, and mobile views.

Advanced Features: The application should reflect the functionality and refinement of a real-world, production-grade web platform. It must extend beyond basic CRUD operations and include at least one advanced, interactive component. Examples include:

- Real-time search, filtering, or sorting (e.g., product search in an e-commerce app)
- Integration with external APIs (e.g., maps, weather, authentication, or payments)
- Dynamic data visualization or interactive dashboards using charts or graphs

Note: These examples are illustrative, not limited — you are encouraged to explore other creative implementations.

3.3 Optional: AI Features – Extra Credit

Teams can earn up to **10 extra points** for implementing meaningful **AI-powered features**. Each team member may contribute one AI feature worth up to **5 points**, for a total of **10 per team**.

- **1 AI feature (one member):** up to 5 pts
- **2 AI features (one per member):** up to 10 pts

Example AI integrations:

- Chatbots or conversational assistants
- AI-driven recommendations or personalization
- Predictive analytics, text classification, or summarization

Students are encouraged to use the **Google Gemini API**, which provides free access to the **Gemini 2.0 Flash model**, suitable for lightweight, fast, and efficient AI integrations such as assistants, recommendation systems, or summarization tools. Other APIs (e.g., OpenAI, Hugging Face, or Cohere) may also be used if they align better with the project's goals.

It is entirely at the team's discretion to use any **paid AI models or APIs**. The course staff will not assume responsibility for related costs. Projects using the free **Gemini 2.0 Flash model** will be considered as meeting all AI integration requirements.

AI features must be original, purposeful, and demonstrably functional. Simply embedding prebuilt outputs without meaningful interaction or logic will not qualify. Extra credit will be awarded during the demo based on creativity, technical implementation, and user impact.

3.4 Technical Requirements

Your application must include:

- **Frontend Technologies:** React (with JSX), React Router
- **Backend Technologies:** Node.js, Express.js
- **Database:** MongoDB or MySQL
- **API Integration:** RESTful endpoints using Fetch or Axios
- **React Hooks:** Must include `useState` and `useEffect`
- **Async Functions:** Use of `async/await` where required for API requests

CRUD Operations (Required) You must implement the following operations for all the pages that your actors interact with (e.g., products, users):

- **POST** – Add new items
- **GET** – Fetch and display list of items
- **PUT** – Update item data (e.g., title, price)
- **DELETE** – Remove item by ID or name or some unique key

3.5 Views and Pages

Your application must contain the following views:

1. **Login and Signup** – User authentication with proper validation
2. **Home or Landing Page** – Introduction and entry point for the app
3. **Detail/Edit Page** – Allows viewing or editing of individual items
4. **Intermediate Process Page** – Simulates a transactional workflow (e.g., checkout)
5. **Confirmation Page** – Display success messages after actions (e.g., form submission)
6. **About Page** – Includes:
 - Course: SE/COM S 3190 – Construction of User Interfaces, Fall 2025
 - Team member names and ISU emails (photos optional, but encouraged)
7. **Company/Client View (Optional)** – If your project was developed for an organization

3.6 Folder and File Structure

Directory Organization:

- `teamXX_final/` – Root project folder
- `frontend/` – React codebase
 - `src/`
 - * `components/`
 - * `assets/`
 - * `App.jsx`
 - * `main.jsx`
- `backend/` – Node/Express server
 - `api/` – API endpoints, Database schema definitions, Logic for each endpoint
 - `config/` (optional) – DB connection files and environment variables
 - `server.js`
- `Documents/` – Architecture PDF, planning files, demo video

3.7 Use of JSON and External Files

Whenever possible, all data must be managed through your project's database. The use of static JSON files should be limited to cases where database integration is not feasible (an exceptional scenario). If used, static data must be stored in external JSON files within the `assets` folder of the `frontend` directory. Do not hardcode data into the application logic. JSON files may be used only for secondary purposes such as dropdown options, UI themes, or demo datasets — not for core database-driven records.

Allowed Libraries and Frameworks

- **CSS:** Tailwind, Bootstrap, custom
- **API:** Fetch, Axios

Note on Penalties and Technical Scope You are encouraged to explore advanced libraries and technologies. However, if the inclusion of external libraries or tools causes the reviewers difficulty in running or evaluating your project, the TA reserves the right to deduct points. Your application must be self-contained and runnable on any standard development machine.

4 Documentation Requirements

4.1 Purpose and Overview

Your documentation serves as the technical narrative of your Final Project. It explains what your system does, how it is structured, and why specific design and implementation decisions were made. The purpose of the documentation is not only to provide a written record of your development but also to help the TA and instructor assess your understanding of web development practices, architecture, and feature integration.

This deliverable is worth 1.5% of your total grade. The document must be clear, comprehensive, and professionally formatted. **If you are familiar with LaTeX, you are strongly encouraged to use it for preparing this documentation.** Overleaf is recommended for its consistent styling, easy formatting of code snippets, and smooth handling of visual elements. However, if you are not comfortable with LaTeX, you may use Microsoft Word or Google Docs, ensuring the final submission is well-formatted and exported as a PDF.

4.2 Format and Submission

- Submit a single PDF file named: `TEAM_NAME_Technical_Report.pdf`
- Push the file to the Documents folder in the GitLab team repository by the Final Project deadline, which is before or at the start of your demo during December 15–18, 2025.

4.3 Suggested Structure

Your documentation should follow a logical structure that presents your project in a way that is easy to follow. Below is a detailed breakdown of the required sections.

4.3.1 Cover Page

- Course code and title: SE/COM S 3190 – Construction of User Interfaces
- Final Project Documentation
- Semester: Fall 2025
- Full names and ISU email addresses of both team members

4.3.2 Table of Contents

Use auto-generated headings to make your document easy to navigate.

4.3.3 Introduction

- A brief overview of the project, what problem it solves, and why it is relevant
- Description of the application’s target users and key features
- State whether your idea is original or inspired by an existing application

4.3.4 Project Description

- Description of all major functionalities
- Explanation of your UI flow — e.g., “Login leads to Dashboard, which allows CRUD operations on Posts”
- Clear mention of your CRUD operations and what entities they apply to

4.3.5 File and Folder Architecture

Required Description:

- Describe the physical layout of your project folder: `frontend/`, `backend/`, `Documents/`
- Mention any static files, external libraries, or helper functions

Optional Diagram: Use a simple folder tree diagram using bullet points or ASCII text (no ER diagrams or database schemas required).

4.3.6 Code Explanation and Logic Flow

- Describe how your frontend interacts with your backend (e.g., through API routes)
- Mention how you use state and props in your components
- Provide 2–3 code snippets (with explanations) that show:
 - A key React component
 - One of each backend route (e.g., POST, GET, PUT, DELETE)
 - Your method of communicating with the database (MongoDB or MySQL)

4.3.7 Web View Screenshots and Annotations

- Include full-page screenshots of each of your views/pages
- Annotate or describe each screenshot with the following:
 - What the page does
 - What user actions are allowed
 - What happens after an interaction (e.g., a successful form submission)

4.3.8 Installation and Setup Instructions

This is one of the most important sections — your application must be runnable.

- Clearly outline steps to run your **frontend/** and **backend/** servers
- Include installation commands for:
 - Node modules (e.g., `npm install`)
 - Database setup (MongoDB local or Atlas, or MySQL setup steps)
- Include commands to start the servers (e.g., `npm run dev`, `nodemon server.js`)
- Mention any `.env` variables needed to run the backend

4.3.9 Contribution Overview

- List all features and who implemented them
- Example:
 - Team Member 1 — Login/Signup, Product CRUD Backend, Tailwind Integration
 - Team Member 2 — Home Page UI, Filters, Confirmation View, Documentation
- Ensure that each student owns both frontend and backend portions

4.3.10 Challenges Faced

- Mention 2–3 specific issues you encountered during development
- Describe how you solved them (e.g., state conflicts, API issues, async bugs)

4.3.11 Final Reflections

- Summarize what you learned through this project
- Reflect on what you might do differently next time
- Optionally, mention whether you would want to deploy this app in the future

4.4 Evaluation Criteria

Your documentation will be evaluated based on:

- Completeness of sections outlined above
- Clarity, readability, and technical explanation
- Quality and structure of the writing
- Proper use of screenshots and visual support
- Setup instructions that work and allow reproducibility

Reminder: Do not include unnecessary technical artifacts such as entity-relationship diagrams, full database schemas, or low-level algorithms. The goal is to show what you built, how it works, and how to run it — all through the lens of applied web development.

4.5 Template

A set of LaTeX and Word/Doc template files will be provided on **December 1, 2025**. You are required to use these templates and fill them in with details specific to your project before submission. The final documentation must be a minimum of **three pages**, excluding the cover page and table of contents.

5 Video Requirements

Your project video serves as a concise and professional walkthrough of your application. It is your opportunity to demonstrate the core features, overall system structure, and user experience in a clear, narrated format. The video helps instructors and TAs quickly assess the functionality and completeness of your project.

This component is worth 1.5% of your final grade. A well-produced video will clearly highlight your application's architecture, interface, and features while demonstrating technical correctness and usability.

5.1 Video Duration and Format

- Length: **3 to 5 minutes**. Videos exceeding this limit may not be fully watched.
- Format: **MP4 only**. Do not use unsupported formats such as .mov, .avi, etc.
- Screen Resolution: At least 720p preferred.

5.2 Recording Guidelines

- **Record your screen using screen capture software** such as OBS Studio, Screenity, QuickTime, or Screencast-O-Matic.
- **Narrate your video using your own voice.** Clearly explain what is being shown on the screen as you walk through the application. **Both team members must actively present the portions of the project they developed**, providing clear explanations of the features, logic, and interactions involved.
- **Mobile phone recordings are not accepted.** Videos must be desktop-recorded to ensure high resolution and clarity.
- Minimize background noise and ensure your voice is clear and audible.
- Multiple takes may be necessary — we encourage you to rehearse and revise.

5.3 Content Checklist

Your video must include the following elements, in logical order:

- **Introduction:** State your name, project name, and what the application does
- **Folder Overview:** Briefly show your folder structure — frontend, backend, and documents

- **Walkthrough Your Code:** Briefly show key portions of your code (e.g., React component, API route, or database call). This part should be concise and should not exceed **1 minute and 10 seconds**.
- **Running the App:** Show both frontend and backend servers running
- **UI Navigation:** Demonstrate all the views/pages (e.g., Home, CRUD Page, Confirmation, About)
- **Data Handling:** Showcase all the complete CRUD operations and how it updates in the database
 - **Note:** Full CRUD operation includes demonstrating POST, GET, PUT and DELETE.
- **Key Feature Highlight:** If you implemented an advanced feature (e.g., file upload and preview, search bar), demonstrate it.
- **Wrap-Up:** Conclude by summarizing the experience or stating that full technical details are provided in the documentation briefly.

5.4 Submission Requirements

- Submit a single MP4 file named: `TEAM_NAME_Final_Video.mp4`
- Push the video into the Documents folder of the GitLab repository before or at the start of your demo during December 15-18, 2025
- File size should not exceed 250MB (compress using HandBrake or Clipchamp if necessary)

5.5 Evaluation Criteria

TAs and instructors will assess your video based on:

- Clarity of narration and video quality
- Logical sequence of content
- Coverage of the required checklist
- Demonstration of a working system and visible CRUD operations
- Overall professionalism and completeness

Reminder: This video is a key part of your evaluation, so take the time to make it clear and professional. It's fine to go slightly over the suggested duration, but not significantly. Treat it as a polished product demo that highlights your application's design, features, and technical depth.

6 Live Demo Presentation Requirements

The live demo is a mandatory, real-time presentation in which both team members must showcase their Final Project to their assigned TA. This component offers an opportunity for students to explain the technical and functional aspects of their project, demonstrate collaboration, and answer questions to validate individual and shared understanding of the system.

This presentation is worth 10% of your final grade. It is a key indicator of your project readiness, technical knowledge, and communication ability.

1. Scheduling and Format

- Demo scheduling details will be shared by your assigned TA via Canvas and Discord during the first week of December. You must schedule your live demo exclusively with your designated TA.
- Scheduling with a TA other than the one assigned to your team will not be permitted. Unauthorized scheduling may result in disqualification from the demo component and a potential score of zero.
- Each team will be allocated a **20 to 30 minute slot**. Slots will be filled on a first-come, first-served basis, so we recommend signing up early.
- All demo presentations will be held **in person**, either in Pearson 0145 or Parks Library, depending on your TA's location. These locations are subject to change. Please check with your TA.
- Teams must arrive on time with their entire application fully set up and operational.
- Please be aware that as this is the Final's Week, some demo sessions may require additional time due to project complexity, troubleshooting, or extended Q&A. If other teams experience technical difficulties or require additional time, it may delay your scheduled slot. Plan accordingly and allocate sufficient time before and after your demo window.

2. Demo Structure and Expectations

The live demo will be divided into 3 main phases:

1. Introduction (2 minutes):

- Briefly introduce the team and the project idea
- State what problem it solves and the core technologies used

2. Live Walkthrough (12-16 minutes):

- Show the full functionality of your application
- Navigate through key views and explain the logic and flow
- Demonstrate all the CRUD sequence end-to-end
- Highlight any advanced feature implemented (e.g., filters, file uploads, dynamic dashboards)
- Show backend console outputs and/or database changes for full-stack validation

3. Q&A and Technical Justification (8 minutes):

- Be prepared to answer detailed questions on:
 - Folder and file structure (`frontend/`, `backend/`, `Documents/`)
 - APIs used and routes implemented
 - Use of React Hooks (`useState`, `useEffect`)
 - Database model(s) and persistence strategy
 - Application setup, configuration, and dependencies

3. Equal Participation is Mandatory

- **Both team members must be present and actively participate in the demo.**
- Each student should clearly explain the portions they implemented.
- Regardless of how tasks were split, **both students must demonstrate full knowledge of the entire application** — frontend, backend, API, and database.
- TAs may ask either student any question, even about features they did not directly code. Full familiarity is expected.

4. Evaluation Criteria

You will be assessed based on:

- Organization and clarity of presentation. Functionality and completeness of the application
- Technical depth of your explanations
- Collaboration and equal participation. Ability to answer technical questions accurately

Reminder: The live demo is a professional interaction. Test your setup in advance, and speak clearly. This session allows the TA to validate that you understand what you built and how it works.

7 Final Project Evaluation Rubrics

7.1 Software Component Rubric

Item	Item Description	Points
Application Functionality	App runs successfully without crashes; routes and actions behave as expected	42
CRUD Implementation	Complete implementation of CRUD for at least two data entities (frontend + backend + DB)	60
UI/UX Design	Interface is responsive, visually cohesive, and user-friendly with a clear interaction flow	42
Routing and Navigation	Use of React Router with proper navigation, fallback views, and seamless transitions	42
React Hooks	Correct usage of useState, useEffect, and any advanced state logic	24
Backend/API Integration	API endpoints are properly implemented and tested using fetch or axios	30
Advanced Feature	A non-trivial feature is included (e.g., filters, file uploads, external APIs, dashboards)	30
Folder/File Organization	Adheres to required structure: <code>frontend/</code> , <code>backend/</code> , <code>Documents/</code>	30
Optional AI Feature	1 feature — 5 pts 2+ features (one per member) — 10 pts	0
Total	Max Possible with Extra Credit 310/300	300

7.2 Technical Report Rubric

Item	Item Description	Points
Formatting	Submitted a PDF that was prepared using typeset or other editor with clean layout and styling	15
Introduction and Project Summary	Clear overview of the project's purpose, users, and goals	15
Architecture Overview	Clear explanation of folder structure, technologies, and integration flow	22
Code Snippets	Meaningful code segments with explanation of functionality	15
Screenshots and View Descriptions	Well-annotated images of application pages	22
Setup Instructions	Steps to run frontend/backend locally with configuration steps	21
Work Contributions	Breakdown of features by team member	15
Retrospective	Challenges faced and key takeaways clearly articulated	25
Total		150

7.3 Video Rubric

Item	Item Description	Points
Length and Format	Video is 3–5 minutes long, in MP4 format	15
Narration	Both team members explain their contributions clearly	30
App Demo	Demonstrates app execution with working views	25
CRUD Flow	Shows at least one complete CRUD workflow	25
Folder Structure	Shows and explains frontend/, backend/, and documents	10
Code Walkthrough	Brief overview of selected code segments within 1 min 10 sec	25
Technical Completeness	Shows outputs, database interaction, and advanced features	20
Total		150

7.4 Demo Rubric

Mini-Assignments contribute directly to your Demo rubric score (100 points out of 300).

Item	Item Description	Points
Presentation Structure	Clear introduction, logical flow, and project explanation by both team members	20
Participation	Equal engagement and communication of each member's contributions	20
System Functionality	Complete walkthrough of app demonstrating stable and interactive features	100
Q&A Readiness	Correctly answers technical questions related to routing, CRUD, folder structure, and config	30
Technical Fluency	Demonstrates deep understanding of the full system architecture and integration	30
Mini-Assignment #1	Initial planning and one working frontend view per member	20
Mini-Assignment #2	React routing and two working views implemented	40
Mini-Assignment #3	Backend API + database integrated and one working CRUD operation	40
Total		300

8 Final Project Points

Component	Points
Proposal	100
Software	300
Documentation & Retrospective	150
Video	150
Demo (including Mini-Assignments)	300
Total	1000

9 Submission Details & Deadline

Deliverable / Event	Due Date / Timeframe
Project Proposal	October 28, 2025
Mini-Assignment #1	November 16, 2025
Mini-Assignment #2	November 23, 2025
Mini-Assignment #3	December 14, 2025
Final Project Code Freeze	December 15-18, 2025 (by demo time)
Final Project Demo	December 15–18, 2025
Final Documentation (PDF)	Before or at your demo (December 15–18, 2025)
Final Video Submission (MP4)	Before or at your demo (December 15–18, 2025)
Grade Release & Review Window	December 19, 2025 – 2:00 PM to 8:00 PM CST

NO LATE SUBMISSIONS will be accepted

Your GitLab repository at the time of the deadline will be considered your final submission. All updates after the deadline will result in penalties.

10 Grade Release and Review Window

Final project grades will be posted on Canvas by 2:00 PM (or earlier) on December 19, 2025. You may contest your grade until 8:00 PM the same day if you believe an error occurred. Use this window wisely and email any concerns clearly. Grades will be finalized and sent to the Registrar soon after.